Tideway: technical documentation

FinTechathon 2026 International Track, Topic E (SME Finance Copilot). Synthetic sandbox only: no real money, bank, lender, supplier or customer is connected.

1. Problem and scope

A small business with payroll approaching and cash short needs bounded, explainable choices, not another dashboard. Tideway turns business records into a source-linked cash gap, plans a recovery, negotiates with simulated counterparties and executes only what a human approved, inside a sandbox. One business uses one currency. The first version targets service businesses and wholesalers with invoices, supplier bills and payroll.

Out of scope: moving real money, production bank integration, binding credit decisions, payroll deferral, a full ERP.

2. Architecture

```
 Browser UI (vanilla JS)  --JSON over loopback, cookie + CSRF-->  server.py (HTTP, CSP,
rate limits, tenant + role checks)
                                                                       |
   conversation.py + i18n.py (small talk, briefings, suggestions, six reply languages)
   tidelm.py (TideLM, on-device fine-tuned model) -> copilot.py (validated read/propose
tools)
   agent.py  (deterministic local agent, optional cloud LLM; read/propose tools only)
                                                                       |
 service.py  -- analysis / propose / negotiate / what-if / ISO 20022 draft / CSV import
     |-- projection.py   exact daily cash ledger, scenarios, look-ahead
     |-- analytics.py    customer profiles, TidePay predictions, seeded Monte Carlo,
KPIs, warnings, payee checks, APR
     |-- ml/             TidePay (simulated federated training, illustrative masking) and
TideLM (data, LoRA config, evaluation)
     |-- planner.py      candidate actions, exact search, robust re-check, explanations
     |-- counterparty.py deterministic simulated suppliers and customers
     '-- store.py        SQLite: workspaces, memberships, invites, approvals, mandates,
executions, audit chain, witness
 security.py (PBKDF2 PIN hashing, HMAC signing, token bucket)   domain.py (validation,
hidden-Unicode, injection scan)
```

The authority boundary is deliberate. The model and the planner live above `store.py`; only `store.py` can approve, sign, execute or change policy, and it checks the caller's role, the mandate signature, expiry and a fresh state hash on every call. Python standard library only; SQLite for storage.

3. Data model and money

Amounts are integers in minor units (`amount_minor`, two decimals). Decimal input is parsed exactly and rejected if it has more than two decimal places, is non-finite, non-positive where positive is required, or above 1,000,000,000,000 units (large enough for high-denomination currencies). If both a decimal and a

`_minor` field are supplied they must agree. Currencies are an ISO 4217 whitelist of 29 two-decimal currencies (USD, EUR, GBP, CNY, SGD, INR, IDR, KES, NGN, AED, BRL, MXN, BDT and more); a business has exactly one, and mixed currencies are rejected. Dates are real `YYYY-MM-DD` calendar dates and the horizon is at most 366 days.

Records: transactions (income/expense), invoices (draft, confirmed, overdue, paid, collected, cancelled, disputed), and bills (open, scheduled, rescheduled, paid, cancelled) with priority (payroll, critical, high, normal, low) and a `reschedule_allowed` flag. CSV import validates the whole file before writing anything (all-or-nothing), rejects unknown or duplicate columns, other tenants' rows, mixed currencies and files over 5,000 rows or 1.5 MB.

4. Projection

`projection.py` builds dated events from open invoices (inflows on expected collection dates), open bills and payroll (outflows) and accepted adjustments, then walks a daily ledger: closing cash is the previous day's cash plus inflows minus outflows. Settled invoices are never counted again. Three scenarios share the engine:

- **Scheduled** (base): the dates in the records.
- **Likely**: each open invoice moves to the customer's expected date from their payment profile.
- **Stress**: a pessimistic (high-percentile) delay per customer.

The planning horizon is the target date plus a look-ahead (default 14 days, 0 to 60), so a plan that only pushes the shortfall past the target date is not counted as a fix. Shapla Services (fixture): opening cash BDT 120,000, payroll 80,000, supplier bill 70,000, confirmed receivable 20,000, reserve floor 20,000. Scheduled minimum cash is **BDT -10,000 on 31 Oct 2026**; the reserve shortfall is **BDT 30,000**.

5. Customer payment profiles

From settled invoices, lateness is `paid_date − due_date`. For each customer the expected delay is a Buhlmann-style credibility blend: own history carries weight n/(n+k) with k = 5, and the business-wide mean carries the rest. The profile reports grade (A to D), confidence (low under 3 settled invoices, medium under 8, high otherwise), on-time rate, p80/p90 delay and a plain-language explanation. Customers with no history fall back to the portfolio, and a business with no history at all to a stated default. A back-test on settled invoices is shown beside the forecast; no accuracy claim is made beyond that synthetic data.

6. Monte Carlo risk

Only payment timing is uncertain. For each of 1,000 runs, one uniform draw per customer (a slow customer is slow on all their invoices) is mapped through TidePay's predicted lateness distribution for each open invoice; overdue invoices only draw delays at least as long as their current lateness; no invoice is paid before its expected date; everything else is fixed. The output is the probability of breaching the reserve floor, of negative cash and of unfunded payroll, expected and p90 shortfall, minimum-cash percentiles, the most common breach date and a fan of daily percentiles. The seed is a hash of business, snapshot hash and target date, and the code uses only `Random.random()` (the stream Python guarantees), so the same records always give the same numbers and tests assert exact results. The UI summarises the result as cash weather: Clear, Unsettled (breach probability of at least 15%) or Storm (a scheduled breach, or at least 50%).

7. Planner (exact, risk-aware)

Candidate actions per record, all within policy:

- **Supplier deferral**: move 50% or 100% of a bill 7 or 14 days later, only if `reschedule_allowed`, never payroll, never a blocked payee. Priced at an assumed 12% a year.
- **Early-payment offer**: offer a customer a 1, 2 or 3% discount to pay earlier; priced as an annualised rate, `d/(1−d) x 365/days` (2% for 20 days is about 3,724 bps).
- **Collection reminder**: a zero-cash draft; a promise to pay is tracked as information, never as cash.
- **Financing**: three labelled MOCK products (invoice advance, working-capital line, guaranteed loan) shown with fee, term and time to cash. Never executable.

Search: every combination of at most one option per record, over the six most relevant records, is evaluated with a fast daily ledger. A bundle is *feasible* if the reserve floor holds on every day of the horizon and payroll is funded; bundles above the daily cap are dropped. Feasible bundles are ranked by shortfall, then cost, then number of counterparties, and the best 40 are re-checked with a seeded 400-run Monte Carlo. The recommendation is the cheapest bundle whose simulated breach probability is within the 10% risk target; otherwise the safest (risk compared in 5-point buckets to absorb sampling noise). Alternatives are chosen to differ in counterparties. If no feasible bundle exists, the explanation says so and escalates. The planner returns structured reasons for every rejected record (for example `SECURITY_BLOCKED`, `DECLINED`).

8. Negotiation with simulated counterparties

`counterparty.py` plays suppliers (styles flexible, standard, cautious, strict) and customers with fixed, repeatable rules and replies **accepted**, **countered** (with terms inside the counterparty's own limits) or **rejected**; a reminder gets an acknowledgement with a promise date. Each reply is labelled simulated. A counter is turned into a revised proposal that needs fresh approval. A rejection excludes that counterparty and triggers a re-plan. Execution requires every action in the mandate to have an accepted reply first.

9. Mandates, approval and execution

1. **Propose**: the agent creates a plan with up to three bundles (recommended and alternatives), each with exposure, risk before and after, and the source records.
2. **Approve** (`store.approve`): the operator re-enters their PIN (PBKDF2, 120,000 iterations; five failures lock the operator for five minutes). Viewers cannot approve. If exposure exceeds the dual-approval threshold, two different people must approve (one of them the owner), and each must type the exact total exposure to show they read it. Approvals made in under two seconds are flagged as possible rubber-stamping in the audit record. The second approval issues the mandate.
3. **Mandate**: a JSON document signed with HMAC-SHA256, modelled on the AP2 mandate idea (cryptographically verifiable, scoped to specific actions, short-lived). It binds actions, amounts, counterparties, plan and snapshot hashes, policy version and a 15-minute expiry. It can be verified offline by pasting the document.

4. **Execute** (`store.execute`): checks the signature, expiry, that the caller is a human approver (or the agent under a standing mandate, for zero-cash reminders only), that the agent is not paused, that all counterparties accepted, and that the underlying records and state hash have not changed since approval. Execution runs in one `BEGIN IMMEDIATE` transaction, stores a receipt with before/after state hashes, then reads the state back. The same idempotency key returns the same receipt; a new key on a consumed mandate is refused.

10. Controls

- **Kill switch / circuit breaker**: pausing blocks execution and planning; only the owner resumes, with a stated reason. Five security denials (structured codes such as `invalid_mandate` or `forbidden_tool`, from signed-in operators only) within ten minutes pause the agent automatically; only denials since the last resume count.
- **Payee fraud**: changed bank details and look-alike names (name similarity of 0.85 or more to a different known payee) block all actions on that bill. Only flagged payees (changed account or look-alike) can be verified: owner-only, by callback to a number on file. Verification clears the block, and whoever verified cannot be the only approver of actions for that payee (separation of duties).
- **Injection and hidden characters**: text is NFKC-normalised, invisible format characters (category Cf) are stripped and flagged; patterns such as "ignore previous instructions", "pay now to this account", "bypass approval", new bank or wallet details (IBAN, SWIFT, PayPal, Wise, Alipay, WeChat Pay, UPI, M-Pesa, Pix, bKash...), markup and links are screened, in English, Chinese, Spanish, French, Indonesian, Hindi, Bangla and Arabic. Suspicious text already in ledger records is quarantined as data and excluded from planning and model tool context. Inbox text is sent only to the local TideLM/TideVision extractor so it can draft a record; the deterministic scan runs alongside it, the result is draft-only, and nothing is saved until a person confirms it.
- **Autonomy dial**: Advise (read and explain), Propose (create plans), Delegate (a standing mandate, seven days, lets the agent send zero-cash reminders, five a day). The matrix is shown in the Security Centre. Moving money and payroll deferral are never delegated.
- **Policy**: reserve floor, per-action limit, daily cap, dual-approval threshold, look-ahead, pause flag. Changes need the PIN.

11. Agent and optional LLM mode

Six tools: `read_ledger`, `profile_customers`, `project_cash`, `simulate_risk`, `check_security`, `compare_plans` (creates a proposal; never approves). Maximum eight tool calls per request. The default **local agent** is a deterministic planner that runs a visible step-by-step rescue run and answers questions from tool output, with no network. The optional **LLM mode** (OpenAI Responses API, user key, per-message consent) sends only pseudonymised counterparty names and tool output, never record free text; tool results are framed as data; every number in the answer is compared against tool output and ungrounded figures are flagged. Hijacked-model behaviour is tested with a simulated transport (see the security self-assessment). No live-model measurement has been made.

12. Audit and evidence

Every proposal, approval, negotiation, execution and rejection is appended to a per-business hash chain (each entry hashes its predecessor). An independent **witness** file with its own key periodically co-signs the chain head, so an attacker who rewrites the database to hide entries, including deleting the tail, is detected.

The Evidence view verifies the chain, shows the first invalid entry and offers a **tamper drill** that edits a copy and shows detection. Exports: evidence JSON, records CSV and receipts CSV (cells beginning `=`, `+`, `-` or `@` are prefixed with `'`), and a draft **ISO 20022 pain.001.001.09** file for deferred supplier payments. The draft is never transmitted.

13. HTTP layer

Loopback only. Host and Origin allow-lists, a per-run CSRF token on every write, `HttpOnly; SameSite=Strict` session cookie (eight hours), JSON-only bodies up to 2 MB, strict field allow-lists per endpoint, role checks, token-bucket rate limits (login stricter), no-store caching and a strict Content-Security-Policy, `X-Frame-Options: DENY`, `nosniff`. Internal errors are logged to the console and not returned.

14. UI

A single-page app: a landing page with sign-up, live demo and join-by-code; then Overview (cash weather, scenarios, fan chart, warnings, read-aloud brief), Rescue (agent run, plan bundles, approval, negotiation, receipt), Assistant (TideLM with its typed plan shown, follow-up suggestions), Inbox (paste messages, confirm drafts), What-if, Records (import, quarantine), Team (members, invites, plan), Security (permission tiers, autonomy dial, kill switch, payee verification, self-test) and Evidence (audit chain, exports, tamper drill).

15. Tideway's own AI

TidePay predicts, for each open invoice, the probability of five lateness bands (on time; 1-7, 8-15, 16-30, more than 30 days late) from 14 features (customer history, amount, terms, seasonality, portfolio pattern). It is a softmax model in pure Python, trained with **simulated federated averaging** across 40 synthetic SMEs. One process holds all synthetic client data and creates reproducible pairwise masks. Global scaling uses pooled training features. This demonstrates aggregation arithmetic, not a distributed cryptographic protocol or a guarantee against a curious server. The historical experiment reported log loss 1.036 versus 1.55 and a small-business subgroup gain; its older split and leaked prior-day baseline must be read with the audit caveats in Section 21. Its distribution drives the likely and stress scenarios and the Monte Carlo. Report: `evidence/ml/tidepay_report.md`.

TideLM is Qwen3-4B-Instruct (4-bit) with a LoRA adapter trained here with MLX on Apple Silicon (rank 16, 16 layers). It has two skills: turning an owner's message in eight languages into a typed tool plan, and turning pasted bank, card and mobile-money alerts and chat messages into draft records while flagging injection. Training data is generated (`tideway/ml/tidelm_data.py`); the test set uses counterparty names never seen in training. Results against the base model are in `evidence/ml/tidelm_report.md`. At runtime (`tidelm.py`), a bounded cache holds scoped prompt prefixes; the request and observed tool results are processed as needed. Model loading can run in the background at start-up. Device verification of the changed protocol remains outstanding. The model cannot mutate money or records directly: plans pass `copilot.validate_plan` (known tools, typed arguments, names resolved against the ledger, read/propose tools only), a forbidden tool counts toward the circuit breaker, replies are composed by code from tool output, and extracted records are drafts until a person confirms them.

16. Conversation and languages

Greetings, thanks, "what can you do?" and "how are we doing?" in many languages are answered instantly from live data with a personal briefing, without calling a model. Every answer carries up to three follow-up questions chosen from the topic and the business's situation, and an unclear question gets examples rather than a dead end. Replies are written in the language of the question (English, Chinese, Spanish, French, Portuguese or Indonesian; Hinglish and Banglish get English) from one phrasebook whose templates take the same numbers in every language, with localised dates; tests check that every message exists in every language with identical placeholders.

17. SaaS workspaces

Each operator has a role per business through memberships; every API call is scoped to a business the operator belongs to, so tenants never see each other's data (403 otherwise), and the configuration lists only the operator's own businesses. Sign-up creates the business, owner account, membership and plan in one transaction, with safety defaults derived from the owner's own cash and reserve. Owners invite people by one-time code (stored hashed, seven-day expiry, plan seat limits) and can remove members but never the last owner. New accounts need a 6-12 digit PIN that is neither repeated nor sequential; sign-up and join share the login rate limiter. Plans are illustrative (Starter, Growth); nothing is charged.

18. Counterparty risk map

For each customer with open invoices, Tideway stresses that customer alone on the likely projection (paying 30 days later than TidePay expects, and never paying) and reports the drop in lowest cash, cash lost by the horizon end and extra days under the reserve. Expected impact weights those losses by TidePay's probabilities, so a reliable large customer and an unreliable small one share one scale. Suppliers show open amounts, next due date, whether deferral is possible and any fraud block.

19. Cross-platform local inference

`local_model.py` supports an optional OpenAI-compatible inference service on Windows, macOS or Linux. It accepts only literal loopback HTTP `/v1` endpoints, rejects credentials/query strings/cloud-labelled model IDs, never follows redirects and caps response sizes. Text and cleaned-photo outputs pass the existing typed tool and draft validators. The UI labels this backend unbenchmarked; it does not inherit TideLM v2 scores. The local service must be trusted and configured to run offline models. See `docs/SETUP_WINDOWS_MAC.md`.

20. Limitations (honest)

Synthetic data; counterparties are rules, not people; the Monte Carlo models payment timing only; the back-test is on synthetic history; PINs and the signing key are demo-grade (a production system needs real identity, hardware-backed keys and a bank rail); the hash chain and witness reduce, but do not eliminate, risk from a fully privileged attacker; TidePay and TideLM are trained and evaluated on synthetic data, so their scores show the method works, not real-world accuracy; no live cloud-LLM or user-study results are claimed; this is not a compliance certification.

21. Contest upgrade: reproducible evidence and linked ledger state

The separate benchmark in `evidence/upgrade-v1/tidepay/` fixes the historical prior's test-label mean leakage and uses global chronological boundaries with label-maturity purges. All fitted statistics use training data; later rolling history includes only payments observable at issue time. On the same 2,376 final-test invoices, federated MAE is 6.9568 days, log loss 1.0727 and sum-over-five-classes Brier 0.5777. Heuristic results are 7.4107 / 1.5688 / 0.7905; local-only 7.2152 / 1.2329 / 0.6407; pooled MAE is slightly better at 6.9448. This audited model is a benchmark artifact; the shipped prediction weights and historical raw evidence are preserved. See `docs/TIDEPAY_EVALUATION_V2.md` for class balance, maturity selection effects, per-business results, confidence intervals and reliability bins. The app links the report/plot publicly; financial APIs retain authentication.

`docs/MODEL_REASONING_V2.md` describes bounded, result-aware local model tool selection. The observable trace separates deterministic preflight from model-selected calls and ties outputs to hashes. Whole-plan contextual validation runs before any proposing tool, with explicit entity clarification, roles, native currency/units, dates and call/output budgets. A safe proposal may require a second model selection after computed cash/risk facts. This is a new runtime protocol, not a rerun of historical TideLM v2 quality scores. The model and weights are absent on the current machine; actual device inference remains outstanding.

`docs/SCENARIO_EVALUATION_V2.md` and `evidence/upgrade-v1/scenarios/` report independently authored ledgers split into development and held-out conditions, labelled benign/attack screening, planner latency and injected-model failure containment. Outcome handling, feasible execution and no-safe-plan escalation have distinct denominators. Screening misses and false positives are retained. No general spreadsheet or generic-LLM comparison is inferred.

`POST /api/demo/payment-delay` is an explicit owner/PIN operation, separate from read-only What-if. It is limited to synthetic demo invoices, validates the whole requested horizon before commit, compares the expected snapshot inside `BEGIN IMMEDIATE`, preserves quarantine, removes a superseded reminder promise, changes the collection date and calls `_bump`. That invalidates pending/approved proposals and live bundle mandates. A unique request id and payload hash replay the same original receipt on retry; conflicts fail. Audit, version and receipt commit atomically. The old approvals remain as evidence; they cannot authorise the changed ledger. A fresh proposal must pass the ordinary approval, signed mandate, simulated negotiation and idempotent execution path. See `docs/PAYMENT_DELAY_RUNBOOK.md` and `scripts/demo_payment_delay.py`.

The tracked `submission-team.public.json` supplies only confirmed public names/institution. Private `submission-team.json`, .env files, databases, signing keys, caches and unrelated root files are excluded from the distributable package. The package builder requires current source hashes in both test and browser reports. `scripts/verify_package.py` verifies distributed bytes, runs extracted tests with Python `-S` and checks fresh startup/public handbook/protected API boundaries without Git or site packages.